

Python Machine Problem Student Supply List Manager

List items, access, update, add, remove, and loop operations in a classroom supply scenario

Field	Details
Activity type	Individual student activity / machine problem
Estimated time	75-100 minutes
Output file	supply_list_manager.py
Allowed references	Class notes and beginner Python references for lists
Important note	This activity sheet intentionally does not include a complete answer or source code solution.

Scenario

Create a Python console program that helps a class adviser manage a small supply checklist for a group activity. The program must start with a given list of items, display selected items, change one item, add one item, remove one item, and print the final list in an organized way.

Learning Targets

- Create and store several related values in a Python list.
- Access list items by position when a specific item is needed.
- Change an existing list item based on the problem instruction.
- Add a new list item based on user input.
- Remove a list item based on user input.
- Loop through a list to display each item clearly.

Required Inputs

- Group name
- Replacement item
- New item to add
- Item to remove
- Display style

Input Options

Input Field	Allowed Option or Format
Group name	Any valid group name
Replacement item	Any supply item that should replace the selected old item
New item to add	Any additional supply item
Item to remove	One item that already appears in the current list
Display style	numbered, plain, or summary

Starting List

Use this exact starting list in the program. This is given data, not the solution code.

```
bond paper, marker, tape, scissors, folder
```

Sample Input Set

Use this only as sample input for testing. It is not the answer and does not show the final code.

Prompt	Sample Input
Group name	Group A
Replacement item	colored paper
New item to add	glue
Item to remove	tape
Display style	numbered

Program Requirements

1. Start the program with the given supply list.
2. Display the first item and the last item from the list.
3. Change one existing item using the replacement item entered by the user.
4. Add the new item entered by the user.
5. Remove the item entered by the user if it is found in the list.
6. Loop through the final list and print each item clearly.
7. Use the selected display style to control how the final list appears.

Problem Conditions

Use these conditions as the basis for your program. Decide your own list operations and loop structure. Do not write the final source code on this sheet.

Condition Area	Given Condition
Starting list	The program begins with five supply items already prepared.
Access items	The output should show the first and last items from the current list before changes are made.
Change item	One old item should be replaced by the replacement item entered by the user.
Add item	The new item entered by the user should appear in the final list.
Remove item	The item selected for removal should no longer appear if it was found.
Loop list	The final list should be printed item by item, not as one unclear block of text.
Display style	The selected style should affect how the final list is printed.
Missing removal item	If the requested removal item is not found, the program should show a short message and continue.

Required Output Format

Your output must be readable and organized. Use this only as a layout guide, not as a code solution.

```
===== CLASS SUPPLY LIST MANAGER =====
Group: <group name>
Starting First Item: <first item>
Starting Last Item: <last item>
Replacement Item: <replacement item>
Added Item: <new item>
Removal Request: <item to remove>
Removal Status: <removed / not found>
Display Style: <style>

Final Supply List:
<items printed one by one>
=====
```

Testing Guide

Run the program using at least three different test cases. The test results must show that the program reacts correctly to changed, added, removed, and looped list items.

Test Case	Suggested Situation	What to Observe
Test 1	User selects numbered display and removes an existing item.	The final list should be numbered and should not include the removed item.
Test 2	User selects plain display and removes a different existing item.	The final list should be printed item by item without numbering.
Test 3	User tries to remove an item not in the list.	The program should show a not-found message and still print the final list.

Sample Testing Scenarios

These samples show possible input and output formats only. They do not show the list operation statements, loop structure, or complete source code answer.

Scenario 1: Numbered Display

Sample Input:

Group name: Group A
Replacement item: colored paper
New item to add: glue
Item to remove: tape
Display style: numbered

Sample Output:

```
===== CLASS SUPPLY LIST MANAGER =====  
Group: Group A  
Starting First Item: bond paper  
Starting Last Item: folder  
Replacement Item: colored paper  
Added Item: glue  
Removal Request: tape  
Removal Status: removed  
Display Style: numbered
```

Final Supply List:

```
1. colored paper  
2. marker  
3. scissors  
4. folder  
5. glue  
=====
```

Scenario 2: Plain Display

Sample Input:

Group name: Group B
Replacement item: cartolina
New item to add: ruler
Item to remove: scissors
Display style: plain

Sample Output:

===== CLASS SUPPLY LIST MANAGER =====

Group: Group B

Starting First Item: bond paper

Starting Last Item: folder

Replacement Item: cartolina

Added Item: ruler

Removal Request: scissors

Removal Status: removed

Display Style: plain

Final Supply List:

cartolina

marker

tape

folder

ruler

=====

Scenario 3: Item Not Found

Sample Input:

Group name: Group C

Replacement item: manila paper

New item to add: pencil

Item to remove: stapler

Display style: summary

Sample Output:

==== CLASS SUPPLY LIST MANAGER =====

Group: Group C

Starting First Item: bond paper

Starting Last Item: folder

Replacement Item: manila paper

Added Item: pencil

Removal Request: stapler

Removal Status: not found

Display Style: summary

Final Supply List:

Total items: 6

Items: manila paper, marker, tape, scissors, folder, pencil

=====

Coding Constraints

- Do not hard-code the sample outputs. They must be produced from the list and user inputs.
- Do not submit a program that only prints fixed text.
- Do not include a complete source code answer in this activity sheet.
- Do not skip checking whether the removal item exists before removing it.
- Do not print the final list as an unclear single object when the chosen style requires item-by-item output.
- Use only beginner-friendly code unless your teacher allows advanced extensions.

Submission Requirements

- Python source file named `supply_list_manager.py`
- At least three screenshots or copied terminal outputs showing different list results
- A short reflection of 3-5 sentences explaining which list operation was hardest to use

Rubric

Criteria	Points	Description
Starting list	10	Creates and uses the required starting list correctly.
Access items	15	Displays selected list items correctly before changes.
Change item	15	Updates one list item based on the user input.
Add item	15	Adds the requested new item to the list.
Remove item	15	Removes the requested item or handles a missing item correctly.
Loop output	20	Prints the final list clearly using a loop or repeated item-by-item display.
Testing evidence	10	Includes multiple test runs that verify different list situations.

Reference Topics

Topic	Reminder
Python lists	Use a list when storing several related values in one variable.
Access list items	Use when a specific item position is needed.
Change list items	Use when an existing value must be replaced.
Add list items	Use when a new value should become part of the list.
Remove list items	Use when an existing value should no longer be included.
Loop lists	Use when each item should be processed or displayed one at a time.